

OneCare NHS FHIR Sandbox Setup & End-to-End Test Guide

1. Obtaining NHS FHIR Sandbox Credentials (Ownership & Access)

The NHS FHIR sandbox credentials (base URL and tokens) are managed as **Vault secrets** at OneCare. Specifically, the credentials reside under the Vault path `kv/health/fhir/orchestrator` and are rotated by the **Integrations** team in coordination with **SRE** ¹. To gain access, you'll need to request it through OneCare's security process – e.g. open a request in the security queue or notify the security/SRE on-call, providing your purpose and how long access is needed ¹. Access is tightly controlled and audited, so ensure to get approval before proceeding.

Rotation & Vault details: FHIR system tokens and client secrets are rotated roughly every **60 days** (or sooner if required by the NHS sandbox) ². The Integrations/SRE owners will update the Vault entry and any dependent systems during each rotation. As a developer, do **not** hard-code these values. Instead, use Vault or an approved password manager (like 1Password) to retrieve the sandbox URL/token. The Vault path above is the source of truth; if you need temporary access (for local dev), an SRE or security engineer might share the token via 1Password (with a time-limited secret) ³. Always follow OneCare's secret handling policies: never commit credentials to git, and only store them locally in a private `.env` file that is excluded from version control ³.

Key contacts and runbooks: The rotation owners are the Integrations team (who handle external FHIR integrations) together with SRE ¹. If you have issues obtaining credentials, reach out in the appropriate internal channel (for example, **#security** or the on-call SRE Slack channel) for guidance. Also check if a runbook exists for "NHS FHIR sandbox access" or consult the *TLS & Credentials* runbook for steps on provisioning external API credentials. Rotation events should be documented in the security runbook, including who rotated the credential and when ⁴ – reviewing that may provide additional context on whom to contact for approvals.

2. Sandbox URL Format & Token – Expected Format and Verification

Once you have access, you should obtain two pieces of information: (a) the **FHIR API base URL** for the NHS sandbox, and (b) an **authentication token** (or the means to generate one) to access the sandbox. These will typically be provided via Vault or 1Password as noted.

- **Sandbox FHIR Base URL:** The URL will likely be an HTTPS endpoint ending in a FHIR path. For example, in development configs it might look like `https://fhir-dev.example.com` or a similar hostname provided by NHS (often with `/fhir` as the base path) ⁵. OneCare's `.env.example` shows a placeholder such as: `FHIR_BASE_URL=https://fhir-dev.example.com` ⁵. Replace this with the actual sandbox URL given (for instance, `https://sandbox.api.nhs.uk/.../fhir`). Ensure you include the correct path up to `/fhir` as instructed by the provider.

- **Auth Token Format (Bearer vs OIDC):** The NHS sandbox typically uses an **OAuth2/OIDC token** for authentication, which you will pass as a Bearer token. In practice, this means your token will be a long JWT string. In OneCare's system, any outgoing calls to FHIR will attach an HTTP `Authorization` header with the value `Bearer <TOKEN>` ⁶. The Vault entry may directly provide a "system token" to use (e.g. a pre-fetched Bearer token) or credentials (client ID/secret) to obtain one. In either case, when configuring your local `.env`, set `FHIR_SYSTEM_TOKEN` (or `FHIR_TOKEN`) to that value ⁵. If the sandbox issues short-lived tokens via OAuth2, you might also receive a `FHIR_CLIENT_ID` and `FHIR_CLIENT_SECRET` ⁵ - in local dev you can manually use these to fetch a new token if needed (or use a provided static token for sandbox testing).

Verification: After configuring the URL and token, it's important to verify you can reach the sandbox outside of OneCare's app. A quick test is to use **cURL** or a similar tool. For example, try fetching the FHIR server's metadata to check connectivity and auth:

```
curl -i -H "Authorization: Bearer $FHIR_SYSTEM_TOKEN" \
"$FHIR_BASE_URL/metadata"
```

A successful response (HTTP 200 with a FHIR CapabilityStatement in JSON or XML) indicates your URL and token are correct. If the metadata endpoint is open, you may get a response without auth; in most cases, however, NHS's sandbox will require the `Authorization: Bearer ...` header for any request. You can also test a simple read, e.g., `GET /Patient` or another resource that the sandbox documentation suggests. If you receive HTTP 401/403, double-check that your token is current and has not expired (or that you included the `Bearer` prefix correctly). OneCare's orchestrator expects all external FHIR URLs to be **HTTPS** for security (more on this in section 6), so confirm your `$FHIR_BASE_URL` begins with `https://`. The application will refuse to use non-HTTPS FHIR endpoints in non-dev modes ⁷.

3. Using a Local Mock FHIR Service (QA Test Environment)

If you cannot immediately get access to the NHS sandbox or need a safe environment for experimentation, OneCare provides a **QA test profile** with a local mock FHIR server. This lets you run the end-to-end stack with a dummy FHIR backend. To use this:

- **Start the QA environment:** In the repository, there is a Docker Compose profile for testing. Run:

```
docker compose --profile test up qa-test-env
```

This command brings up the minimal set of services needed for an end-to-end test, including a fake FHIR service on a known port (likely **9500**). The compose file will label this service (e.g. `qa-test-env` or similar) and expose an endpoint that mimics the FHIR API.

- **Seed the FHIR data:** Once the containers are up, run the provided seed script to populate the mock FHIR server with initial data:

```
./qa/env/seed.sh
```

This script will load sample resources (patients, slots, etc.) into the mock FHIR service so that the orchestrator can read/write as if it were a real NHS system. Check the output to ensure seeding succeeds (the script may log created resource IDs or confirmations).

- **Orchestrator configuration:** The orchestrator in test mode is configured to target the local FHIR service automatically. In other words, when running with this `test` profile, the orchestrator should have its `FHIR_BASE_URL` pointed at `http://localhost:9500/fhir` (or the Docker network alias) by default. The platform's QA documentation indicates that the orchestrator "auto-resolves" the FHIR URL to the local stub when using the hermetic test environment ⁸. This means you shouldn't need to manually change `.env` for FHIR settings when using the test profile – just ensure the profile is active so the orchestrator knows to use the local stub.
- **Usage and endpoints:** The mock service will listen on port 9500 and expose a FHIR API. The orchestrator will perform the same operations (create Task, Appointment, etc.) against `http://localhost:9500/fhir` as it would against the real NHS sandbox. You can verify the mock is receiving requests by checking its logs (the Docker container logs should show FHIR resource accesses when you exercise the system). Keep in mind this local FHIR is only for testing – **no PHI** should be used here since it's just seeded dummy data.

This self-contained QA environment is very useful for initial development and troubleshooting. It was designed to be **hermetic and reproducible** ⁸ – meaning each developer can spin up the same deterministic environment. The accompanying documentation (see `docs/QA_ENV.md` if available) describes the available environment variables, endpoints, and how to run end-to-end tests in this mode ⁸. In summary, if the real NHS sandbox is not accessible or you're writing tests (CI or local), use this Docker profile to stand up a local FHIR API at `localhost:9500/fhir` and seed it with the necessary data for OneCare's stack.

4. Local Environment Configuration & Running the Stack

With credentials and FHIR endpoint in hand (or after setting up the local mock), you can configure your `.env.dev` file and start the OneCare stack on your machine. Key things to set and check:

- **Environment Variables (.env.dev):** Copy `.env.example` to a new file (e.g. `.env.dev` or simply `.env.local`) and fill in the placeholders. At minimum, set the following:
- `FHIR_BASE_URL` – The base URL to the FHIR API. Use the sandbox URL (`https://.../fhir`) or `http://localhost:9500/fhir` if using the local mock ⁵.
- `FHIR_SYSTEM_TOKEN` – The Bearer token for the sandbox. Insert the value retrieved from Vault/1Password (make sure not to quote it, just paste the token string) ⁵.
- (If provided) `FHIR_CLIENT_ID` and `FHIR_CLIENT_SECRET` – If the sandbox requires OAuth client credentials, include these as well ⁵. Otherwise, leave them as placeholders or remove if not used.
- `PRACTICE_ID` – Set this to identify which practice config to load. For development, you can use `demo` (the orchestrator will default to a built-in "demo" practice if none is set) ⁹. In fact, `demo` is used for the example config (NHS GP default settings). Alternatively, if you have a specific practice code for testing (say "p1" corresponding to a test org), you could use that. **Note:** The orchestrator *requires* this env var in non-test modes – if not set, it will throw an error on startup ¹⁰.

- Other env vars: ensure `NODE_ENV=development` (or omit it for default), and that ports are configured if you need non-default. The defaults are usually fine (`PORT_ORCHESTRATOR=3001`, etc.). If you are running the Python services (Safety Gate and Scribe) locally, confirm `PY_SAFETY_GATE_URL=http://localhost:8081` (and similarly for Scribe on 8082) to point orchestrator to the local ML services ¹¹.

- **Port Conflicts:** By default, the services will use **port 3001** for the Orchestrator (Node service) and **8081** for the Safety Gate (Python FastAPI service) ¹¹ ¹². Make sure nothing else on your system is occupying these ports. If you have a development server or another app running on 3001 or 8081, stop it or change the ports to avoid conflicts. The portal (frontend) if running, might use port 3000 – typically that's fine, but our focus here is the backend services. Also note that the Docker-based NATS server (if you use Docker Compose) listens on 4222 and 8222; those rarely conflict but be aware if you have another NATS or service on those.

- **Starting the stack (local dev mode):** OneCare provides a convenient Makefile and NPM scripts to run everything. For example, you can simply run:

```
make dev-run
```

This will build and launch the core backend: it starts the Safety Gate ML service on port 8081, then the Orchestrator on port 3001 (after building it) ¹³. Under the hood, it's equivalent to manually doing what the usage guide suggests: launching the Python app and Node app concurrently. If you prefer manual steps or need to debug startup issues, you can do:

- **Run Safety Gate** – e.g., `uvicorn services-py/safety_gate_service/main:app --reload --port 8081` ¹⁴. This starts the FastAPI server for triage.
- **Build and run Orchestrator** – e.g., `npm -w @onecare/app-orchestrator run build` to compile it, then `PORT=3001 node apps/orchestrator/dist/index.js` to start the service ¹⁴. (Ensure the env vars from your `.env.dev` are loaded so it knows the FHIR URL, token, etc. Using a tool like `direnv` or `npm run dev` script that sources `.env.dev` can help.)
- *(Optional)* Run Scribe – if you need the ambient scribe service (port 8082) for any reason, you can similarly start it via `uvicorn` (though for basic FHIR integration testing, Scribe isn't needed).

Another approach is `npm run dev:all` if available – this might use a utility like **concurrently** to start multiple services in one command. Check the project package scripts for a `dev:all` or similar. The Makefile covers most cases with targets like `py-safety`, `py-scribe`, and `dev-run` ¹⁵ ¹³.

- **Readiness and health checks:** After starting the services, verify they are up and connected. The orchestrator exposes health endpoints:
- **Health:** `GET http://localhost:3001/health` should return a simple OK (HTTP 200) if the service is running ¹⁶.
- **Readiness:** `GET http://localhost:3001/ready` will return HTTP 503 until all critical connections are live (e.g., it waits for the message bus/NATS to connect) ¹⁶. In local memory-bus mode, it should turn healthy almost immediately.
- The Safety Gate has a docs endpoint at `http://localhost:8081/docs` (FastAPI swagger UI) to confirm it's running ¹⁶.

You can use `curl` or simply check the console logs. OneCare's startup log will print a line that the practice config was applied (showing practiceId and some config values) ¹⁷ ¹⁸, and logs for

connecting to NATS (if using it) or using in-memory bus. If anything is misconfigured (like a missing env), the service may exit with an error – read the message closely (e.g., forgetting PRACTICE_ID will throw an error telling you it's required ¹⁰). Fix any such issues and restart.

- **Portal (optional):** If you want to run the frontend Portal to manually exercise the flows, you can run `npm start` or `npm run dev` in the `apps/portal` workspace. Ensure that `VITE_BOOKING_API_URL` in your env points to the orchestrator (e.g., `http://localhost:3001`) so the portal calls hit your local orchestrator ¹⁹. Also set `BOOKING_AVAILABILITY_URL` to the upstream booking API if you want live slot data – in local dev you might leave this as the default or use the dummy as in `.env.example` ²⁰. The portal will then open at `http://localhost:3000`. However, using the API via curl or tests (next section) is sufficient for back-end verification if you don't need the UI.

5. End-to-End Validation (API Calls and Tests)

With the stack running (either against the sandbox or the local mock), you should validate that OneCare's orchestrator can perform the key operations: a safety check (triage) and a booking flow. There are a few ways to do this: calling the APIs directly, running the automated end-to-end tests, or using built-in smoke tests.

- **Calling `/safety-check` API:** This endpoint triggers the triage flow (Portal intake through Safety Gate and back). You can simulate a patient submission with a simple JSON payload. For example, using curl from your development machine:

```
curl -X POST http://localhost:3001/safety-check \
-H 'content-type: application/json' \
-H 'authorization: Bearer <YOUR_TOKEN>' \
-H 'x-actor-type: patient' \
-H 'x-actor-id: demo-patient' \
-H 'x-request-id: req-12345' \
-H 'x-auth-scope: submit triage:submit' \
-d '{
  "practiceId": "p1",
  "patient": { "id": "abc" },
  "narrative": "I have chest pain",
  "channel": "web"
}'
```

In this example (which mirrors the Makefile's demo command ²¹), replace `<YOUR_TOKEN>` with a valid token. The headers here are critical: OneCare's orchestrator uses a zero-trust approach that **requires** every request to include an `Authorization` header and several custom headers ⁶. Specifically: - `Authorization: Bearer ...` – the token (for now, you can use the sandbox system token here; in a real scenario this might be a patient's auth token, but the system is not fully enforcing distinct user tokens in dev). Using the sandbox token is acceptable for testing as a system-level token. - `X-Actor-Type` – who is making the request. For a patient submission, use `patient`. (Other valid types are `practitioner` or `system` for different contexts ⁶.) - `X-Actor-Id` – an identifier for the actor. In dev, you can use a dummy like "demo-patient" or an NHS number/identifier if required. This just tags the request for logging/audit. - `X-Request-Id` – a unique ID for the request (use any random GUID or string). This helps with idempotency and logging. - `X-Auth-Scope` – the scopes or permissions associated with the token/actor. For a safety-check (triage) call, include the scope for triage

submission. The example above shows `submit triage:submit` ²². It appears the system expects a scope string containing at least `triage:submit`. (The word “submit” before it might be a general action scope – the exact format can be seen in the example and the code expects `triage:submit`.)

The JSON body includes `practiceId` (use the practice you set in env or “p1” as in examples), a minimal patient object with an `id`, a narrative (the complaint text), and channel. If everything is configured, the orchestrator will forward this to the Safety Gate service and return a decision. A successful response will look like a `SafetyDecision` object (with fields like `outcome` such as `SAFE_TO_PROCEED` or needs escalation, etc., plus a `correlationId`). If any header is missing or token is wrong, you'll get a 401/403 with an error code. OneCare's error envelopes include a code and `correlationId` for debugging ²³ ²⁴. For example, missing auth would yield code `unauthorized` (401) or `forbidden` (403) depending on what's wrong.

- **Calling `/booking` APIs:** Once you have a `SAFE_TO_PROCEED` decision from the safety-check, the next part is booking an appointment slot (if using the GP Connect integration). The orchestrator (or the booking microservice) exposes endpoints under `/booking/*` for this. Specifically:

- **Search available slots:** `GET http://localhost:3001/booking/slots?serviceType=<type>&from=<start>&to=<end>&...` – this retrieves available appointment slots. The query parameters can include `serviceType` (e.g., a slot type or clinic type), a time window (`from` and `to` datetimes), and possibly an organization or location filter. In the Portal UI, these correspond to user selection of a timeframe and modality. When calling this endpoint, include the same headers as before: `Authorization` and the `X-Actor-*`, `X-Request-Id`. Additionally, for booking operations the system expects the scope `booking:read` on the auth scope ²⁵ (since searching slots is a read action). The test suite demonstrates an example header set for booking slot queries: it uses `authorization: Bearer token` (use your token), `x-actor-type: patient`, `x-actor-id: <patientId>`, and crucially `x-auth-scope: booking:read` ²⁵. Also include `x-patient-id` header (the patient's ID) if available ²⁶ – this might be used internally for consent checks. If your request is well-formed, the response will be a list of available `Slot` resources (or an array of simplified slot objects). The Portal's logic will then let the patient pick one.

- **Confirm a booking (create appointment):** Once a slot is selected, the client will call `POST http://localhost:3001/booking/appointments` with a JSON body like `{"slotId": "...", "patientId": "...", "reason": "some reason"}`. To simulate this, you'd do a POST with similar headers as above. For booking creation, likely the scope needed is still `booking:read` or possibly `booking:write` – however, in the code, they treat any booking operation under the same scope guard (the constant is `BOOKING_SCOPE = 'booking:read'` for both) ²⁷. Include an `X-Idempotency-Key` header as well on this request ²⁸. The frontend generates an idempotency key to prevent duplicate booking requests (you can create one by concatenating the `slotId`, `patientId`, and a timestamp, or simply use a UUID for testing). The orchestrator will use that to ensure it doesn't double-create an appointment if the call is retried. With all headers in place (`Authorization`, `X-Actor-Type/Id`, `X-Request-Id`, `X-Auth-Scope: booking:read`, and `X-Idempotency-Key`), the POST to `/booking/appointments` will attempt to create the appointment via GP Connect. In the sandbox or local mock, this should succeed if the slot is still free. The response on success will contain details of the booked appointment – e.g., an `appointmentId` and possibly the slot info and a `correlationId` ²⁹ ³⁰. On failure (slot taken, etc.), you'd get an error code (`conflict` for double-booking, etc.) with a corresponding HTTP 409 or similar ³¹ ³².

Because these calls are a bit complex to craft by hand, an easier way to test end-to-end is to use the automated tests or provided scripts:

- **Automated E2E tests:** Run `npm run test:e2e`. The repository's test suite (likely using Vitest or Jest) includes end-to-end tests that spin up the app in a controlled environment and simulate these flows. For instance, there are tests for booking slot validation³³ and presumably for the full booking flow. Before running, ensure you've built the project (`npm run build`) and that any required services are running or can be started (some tests may start an in-memory server automatically as we saw). The tests might stub external calls (e.g., they stub out the security checks and use a dummy FHIR/GP Connect client), so they should pass as long as the code is working. You might need to set `NODE_ENV=test` and have the environment variables (like `FHIR_BASE_URL`, tokens, etc.) available if the tests require them. Check the **README or docs for testing** for any environment setup. For example, the test we saw sets `BUS_IMPL=memory` and a dummy `BOOKING_AVAILABILITY_URL` before starting the server³⁴, so the tests handle a lot automatically.
- **Smoke test / performance harness:** The Makefile includes some handy targets to simulate calls. You can use `make stack-smoke` after your stack is up, which will do a quick health check and then fire a sample `/safety-check` call automatically³⁵. This is a quick way to see if the basic triage flow is working end-to-end (it uses the same demo patient and dev-token as earlier example). For performance or load testing, there are scripts under `scripts/perf/` (e.g., `booking_flow.js`, `orchestrator.sh`). One target is `make perf-orchestrator` which runs a small load test against the `/safety-check` endpoint (10 concurrent requests by default)³⁶. This can be useful to verify stability. Likewise, `npm run test:perf` or similar might exist. Use these with caution – ensure your machine can handle it and you're not hitting the real NHS sandbox with load beyond their allowed limit (if your `FHIR_BASE_URL` is pointed at sandbox, keep load minimal).
- **Portal UI click-through:** As a final validation, if you run the React portal on <http://localhost:3000>, you can go through the "Intake" form and then "Booking" UI. The portal will call the same `/safety-check` and `/booking/slots` endpoints via the orchestrator (remember to log in or satisfy any auth the portal might require; in dev, it might not have a login and just use the backend with a default token/cookie). Watching the network calls in your browser dev tools can confirm that each piece is working. This is more of a sanity check since the programmatic tests above already hit the APIs.

6. Security & Compliance Reminders (PHI, HTTPS, Token Hygiene)

Finally, as you work with NHS data and OneCare's system, keep in mind the security and compliance requirements:

- **Protected Health Information (PHI) Handling:** All clinical data and patient-identifiable information must be handled carefully. OneCare's logging and data policies ensure no sensitive info is exposed. As a developer, follow these guidelines:
- **Do not log PHI/PII** – e.g., never print patient names, narratives, or any identifying info in logs. The orchestrator's code explicitly redacts or omits such fields. Recent tasks included ensuring that narrative text and other PHI are removed from logs and replaced with stable IDs or nothing³⁷. If you add any logging, use the existing logger which has redaction built-in³⁸, and log only high-level events or IDs.

- **Minimize data in transit** – when the orchestrator publishes events or sends data to other services, it should only send the necessary identifiers. The system is designed to send references (like FHIR resource IDs) over the bus, rather than entire documents, to limit PHI exposure ³⁷. Adhere to this pattern if you modify any integration code.
- **Retention and consent** – The NHS sandbox data still should be treated as real PHI for compliance. Follow any data retention policies (OneCare’s config may set `retention_days`, etc.). Do not store sandbox data outside approved systems. Also ensure any patient data you use for testing has appropriate consent (in practice, use synthetic or non-identifiable data for tests).
- **HTTPS enforcement:** OneCare requires secure connections to any external health API. The orchestrator will **enforce HTTPS for the FHIR base URL** except in explicit dev/test scenarios. As part of SSRF protection and to meet NHS security requirements, the code checks that configured URLs are not pointing to private IPs or insecure protocols ³⁹. In practice, this means if you accidentally set `FHIR_BASE_URL` to an `http://` address in a non-dev environment, the service will reject it and log a warning or error. (In dev you can use `http://localhost` for the stub, but in staging/prod it must be `https://...`). This “`assertHttpsUrl`” check (as it’s informally called) is there to ensure no data is sent in plaintext ⁷. Additionally, the TLS runbook covers certificate validation – e.g., if the NHS sandbox uses a custom CA, you might need to configure the cert bundle. By default, the Node and Python clients will validate SSL certs; do not disable SSL verification unless absolutely necessary for a local mock. Always use the official sandbox hostname with HTTPS.
- **Token security & storage:** Treat the sandbox auth token as sensitive. Do not embed it in source code or share it in chat. Load it via env var or Vault at runtime ³. When running locally, avoid leaving it in your shell history (if sourcing manually, use a `read -s` to hide input, etc., as noted in OneCare’s developer workflow guidelines ³). Also, OneCare’s logging has been designed to **never print the Authorization header or tokens** – the code actively redacts tokens in logs for security ³⁷. If you see any token in logs, that’s a bug – report it. On your local machine, if you use debugging or print statements, be cautious not to accidentally log the token. After use, if you wrote down or copied the token somewhere, clear it out (for example, if stored in a temporary file or notes).
- **PHI in transit (HTTPS) and at rest:** Ensure that when you call the sandbox, you do so over HTTPS (which you will, given the URL). The `assertHttpsUrl` enforcement and the platform’s networking policies ensure compliance with NHS data handling rules by not allowing unencrypted channels ⁷. If you ever capture request logs or use a proxy for debugging, treat those traces as sensitive because they might contain PHI.
- **Cleaning up after testing:** The sandbox data might contain PHI, and any PHI you input (like test patient details) should be fictitious. Nonetheless, do not leave any exported data lying around. If you performed database dumps or enabled verbose logs for debugging, handle those according to OneCare’s guidelines (e.g., secure storage or deletion).

In summary, security is paramount: use strong TLS, assume responsibility for any PHI (even in a sandbox), and keep secrets out of source code. The platform has multiple safeguards (token redaction, allow-lists for outbound calls, replay guards, etc.) ³⁹ ³⁷ – make sure your usage of the system doesn’t circumvent those. By following the above steps and precautions, you’ll spin up OneCare’s stack against the NHS FHIR sandbox successfully and in compliance with all security requirements.

Sources:

1. OneCare Vault & Security Policies – *Secrets management and rotation* ¹ ²
2. OneCare `.env.example` – *FHIR config placeholders* ⁵
3. OneCare Orchestrator Guide – *Required headers and practice config* ⁶ ⁹
4. OneCare QA Task – *Hermetic test env with seeded data* ⁸
5. OneCare Makefile – *Local run commands and curl examples* ²¹ ¹³
6. OneCare Test Suite – *Booking API usage and headers* ²⁵
7. OneCare Security Development Guidelines – *Secret handling and logging* ³⁷ ³⁹

¹ ² ³ ⁴ SECURITY.md

<https://github.com/georgeahmed/onecare/blob/766e2637d808d0f8b268d7d4bf23ffc53d6c340d/docs/SECURITY.md>

⁵ ¹¹ ¹⁹ ²⁰ .env.example

<https://github.com/georgeahmed/onecare/blob/766e2637d808d0f8b268d7d4bf23ffc53d6c340d/.env.example>

⁶ ⁹ README.md

<https://github.com/georgeahmed/onecare/blob/766e2637d808d0f8b268d7d4bf23ffc53d6c340d/apps/orchestrator/README.md>

⁷ ³⁹ IN-01.8.md

<https://github.com/georgeahmed/onecare/blob/766e2637d808d0f8b268d7d4bf23ffc53d6c340d/team/integrations/tasks/IN-01.8.md>

⁸ QA-01.15.md

<https://github.com/georgeahmed/onecare/blob/766e2637d808d0f8b268d7d4bf23ffc53d6c340d/team/qa-automation/tasks/QA-01.15.md>

¹⁰ ¹⁷ ¹⁸ ²⁷ index.ts

<https://github.com/georgeahmed/onecare/blob/766e2637d808d0f8b268d7d4bf23ffc53d6c340d/apps/orchestrator/src/index.ts>

¹² ¹⁴ ¹⁶ USAGE.md

<https://github.com/georgeahmed/onecare/blob/766e2637d808d0f8b268d7d4bf23ffc53d6c340d/docs/USAGE.md>

¹³ ¹⁵ ²¹ ²² ³⁵ ³⁶ Makefile

<https://github.com/georgeahmed/onecare/blob/766e2637d808d0f8b268d7d4bf23ffc53d6c340d/Makefile>

²³ ²⁴ ERRORS.md

<https://github.com/georgeahmed/onecare/blob/766e2637d808d0f8b268d7d4bf23ffc53d6c340d/docs/ERRORS.md>

²⁵ ²⁶ ³³ ³⁴ bookingSlotsValidation.test.ts

<https://github.com/georgeahmed/onecare/blob/766e2637d808d0f8b268d7d4bf23ffc53d6c340d/apps/orchestrator/test/bookingSlotsValidation.test.ts>

²⁸ api.ts

<https://github.com/georgeahmed/onecare/blob/766e2637d808d0f8b268d7d4bf23ffc53d6c340d/apps/portal/src/lib/api.ts>

²⁹ ³⁰ ³¹ ³² gpconnect.client.ts

<https://github.com/georgeahmed/onecare/blob/766e2637d808d0f8b268d7d4bf23ffc53d6c340d/apps/booking/src/adapters/gpconnect.client.ts>

³⁷ ³⁸ BE-01.20.md

<https://github.com/georgeahmed/onecare/blob/766e2637d808d0f8b268d7d4bf23ffc53d6c340d/team/backend/tasks/BE-01.20.md>